

Esercizio 3 — Buildweek 3

Team Akatsuki • Domande e Risposte sulla Gestione Filesystem e Permessi

D: Qual è il significato dell'output? Dove sono fisicamente memorizzati i file elencati? Perché `/dev/sdb1` non viene mostrato nell'output sopra?

R: L'output del comando `ls -l` mostra la struttura dei file e delle directory presenti nella root directory (`/`) del sistema operativo Linux.

Significato dell'output:

- **Struttura FHS (Filesystem Hierarchy Standard):** Vengono elencate le directory principali del sistema (ad esempio `/etc` per i file di configurazione, `/home` per le cartelle utente, `/bin` per gli eseguibili di sistema).
- **Autorizzazioni e proprietà:** La prima colonna mostra i permessi (es. `drwxr-xr-x`), seguiti da proprietario e gruppo (tutti appartenenti a `root`).
- **Link simbolici:** Diverse cartelle come `bin`, `lib` e `sbin` mostrano una freccia (`-> usr/bin`), indicando che sono collegamenti simbolici (symlink) che puntano alle reali directory sotto `/usr`.

Dove sono fisicamente memorizzati i file?

- **Memoria di massa (Hard Disk / SSD):** Directory come `/etc`, `/usr`, `/var`, `/home`, `/opt`, `/boot` e `/root` contengono file persistenti memorizzati nel file system del disco fisso (la partizione montata su `/`).
- **Memoria volatile (RAM / Kernel in memoria):**
 - `/proc` e `/sys`: Non contengono file reali su disco, ma file system virtuali generati dinamicamente dal kernel Linux in memoria RAM per mostrare informazioni sui processi e sullo stato del sistema.
 - `/dev`: È un file system gestito da udev (sposto un tmpfs o devtmpfs), residente in RAM, che espone i nodi di dispositivo gestiti dal kernel.
 - `/run` e `/tmp`: Spesso montati come tmpfs, risiedono nella memoria RAM (i file contenuti al loro interno vengono persi al riavvio).

Perché `/dev/sdb1` non compare nell'output?

`/dev/sdb1` non compare per due motivi principali:

1. **Non si trova in `/`:** Il comando `ls -l` è stato eseguito nella directory radice (`/`). `/dev/sdb1` è un file speciale (nodo di dispositivo) situato all'interno della directory `/dev`, non direttamente sotto `/`. Se esegui `ls -l /dev`, vedresti apparire `sdb1`.
2. **È un file di dispositivo, non una directory montata:** `/dev/sdb1` rappresenta l'interfaccia a basso livello per accedere alla prima partizione del secondo disco fisso (sdb). Perché il contenuto di un disco sia visibile nel file system, la partizione deve essere montata in un punto di mount specifico (come `/mnt`, `/media` o `/home`).

D: Perché la directory non è più vuota? Dove sono fisicamente memorizzati i file elencati?

R:

1. Perché la directory non è più vuota?

Prima di eseguire il montaggio, la cartella `second_drive` era semplicemente una directory vuota creata sul tuo disco principale (dove risiede `/home`).

Con il comando `mount /dev/sdb1 ~/second_drive`, ho "sovrapposto" il contenuto del file system di `/dev/sdb1` a quella cartella. La directory `second_drive` ora fa da punto di mount (finestra d'accesso): invece di mostrare lo spazio vuoto sul disco principale, mostra tutto ciò che era già presente o salvato all'interno del disco/partizione `/dev/sdb1` (in questo caso, il file `myFile.txt` e la cartella di sistema `lost+found`). Smontando la partizione con `umount ~/second_drive`, la cartella tornerà a essere vuota come prima.

2. Dove sono fisicamente memorizzati i file elencati?

I file (`myFile.txt` e `lost+found`) sono memorizzati fisicamente nei blocchi di memoria del secondo disco fisso/SSD, cioè il dispositivo hardware corrispondente a `/dev/sdb1`.

Non occupano spazio sul disco principale dove risiede la tua home directory. La cartella `/home/analyst/second_drive` è solo un collegamento logico: la lettura e la scrittura dei dati avvengono direttamente sui settori fisici dell'unità secondaria sdb.

D: Considera il file `cyops.mn` come esempio. Chi è il proprietario del file? E il gruppo?

R: Proprietario: `analyst` | Gruppo: `analyst`

D: I permessi per `cyops.mn` sono `-rw-r--r--`. Cosa significa?

R:

- Il primo carattere (`-`) indica che si tratta di un file regolare (directory = `d`, link = `l`).
- `rw-` (Proprietario): permessi di lettura e scrittura, nessuna esecuzione.
- `r--` (Gruppo): solo lettura.
- `r--` (Altri utenti): solo lettura.

D: Perché il file non è stato creato? Elenca i permessi, la proprietà e il contenuto della directory `/mnt` e spiega cosa è successo.

R:

```
drwxr-xr-x 2 root root 4096 Jan 5 2018 /mnt
```

L'utente `analyst`, non essendo il proprietario della cartella e non appartenendo al gruppo con permessi elevati, non possiede i permessi di scrittura all'interno della directory `/mnt`.

D: Cosa si può fare affinché il comando `touch` mostrato sopra abbia successo?

R: Si può eseguire il comando tramite privilegi di amministratore usando `sudo` oppure si possono modificare i permessi della directory di destinazione tramite il comando `chmod` (o cambiarne la proprietà con `chown`).

D: Quali sono i permessi del file `myFile.txt` ?

R:

```
-rw-r--r-- 1 analyst analyst 183 Mar 26 2018 myFile.txt
```

D: Quali sono i permessi del file `myFile.txt` dopo `chmod 665` ?

R:

```
-rw-rw-r-x 1 analyst analyst 183 Mar 26 2018 myFile.txt
```

D: Quale comando cambierebbe i permessi di `myFile.txt` a `rwxrwxrwx`, garantendo a qualsiasi utente nel sistema pieno accesso al file?

R: `chmod 777 myFile.txt`

D: L'operazione è riuscita? Spiega.

R: Sì, l'operazione è riuscita grazie alla modifica corretta dei permessi di accesso.

D: Qual è la differenza tra la parte iniziale della riga di `malware` e la riga di `mininet_services` ?

R: La `d` iniziale presente nella riga di `mininet_services` indica che si tratta di una directory, mentre l'assenza della `d` (o la presenza di `-`) nella riga di `malware` indica che si tratta di un file regolare.

D: Cosa pensi succederà a `file2hard` se aprissi un editor di testo e cambiassi il testo in `file2new.txt` ?

R: Null'altro che l'aggiornamento reciproco: `file2hard` è un hard link (o punta allo stesso inodo/contenuto). Modificando il contenuto di `file2new.txt`, leggendo con `cat file2hard` si vedrà il contenuto aggiornato di `file2new.txt`, poiché entrambi i riferimenti puntano ai medesimi dati fisici sul disco.